

REST API Mini Project : Employee Management System

Day 10 Assignment

Step 1 : install Node

```
C:\Users\pathi\Downloads\StudentRestAPI>node -v
v22.7.0

C:\Users\pathi\Downloads\StudentRestAPI>
```

Step 2 : Install JSON Server

```
C:\Users\pathi\Downloads\StudentRestAPI>npm install -g json-server

changed 119 packages in 16s

22 packages are looking for funding
run 'npm fund' for details
```

Step 3 : Create db.json file

```
db.json U X
C:\Users\pathi\Downloads\StudentRestAPI>db.json > {} employees > {} 4
1
2 {
3   "employees": [
4     {
5       "id": 1,
6       "empId": 101,
7       "firstName": "John",
8       "lastName": "Doe",
9       "email": "john.doe@gmail.com",
10      "mobileNumber": "9876543210"
11    },
12    {
13      "id": 2,
14      "empId": 102,
15      "firstName": "Sai",
16      "lastName": "Kumar",
17      "email": "sai.kumar@gmail.com",
18      "mobileNumber": "9123456789"
19    },
20    {
21      "id": 3,
22      "empId": 103,
23      "firstName": "Abhi",
24      "lastName": "Sharma",
25      "email": "abhi.sharma@gmail.com",
26      "mobileNumber": "9988776655"
27    },
28    {
29      "id": 4,
30      "empId": 104,
31      "firstName": "Priya",
32      "lastName": "Reddy",
33      "email": "priya.reddy@gmail.com",
34      "mobileNumber": "9012345678"
35    },
36  ]
37 }
```

Step 4 : Run the JSON Server to watch the db.json file

```
C:\Users\pathi\Downloads\StudentRestAPI>json-server --watch db.json --port 3000

\{^_^}/ hi!

Loading db.json
Done

Resources
http://localhost:3000/employees

Home
http://localhost:3000

Type s + enter at any time to create a snapshot of the database
Watching...
```

GET Method to Get all Employees

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/employees`. The response is a 200 OK status with a JSON body containing two employee records.

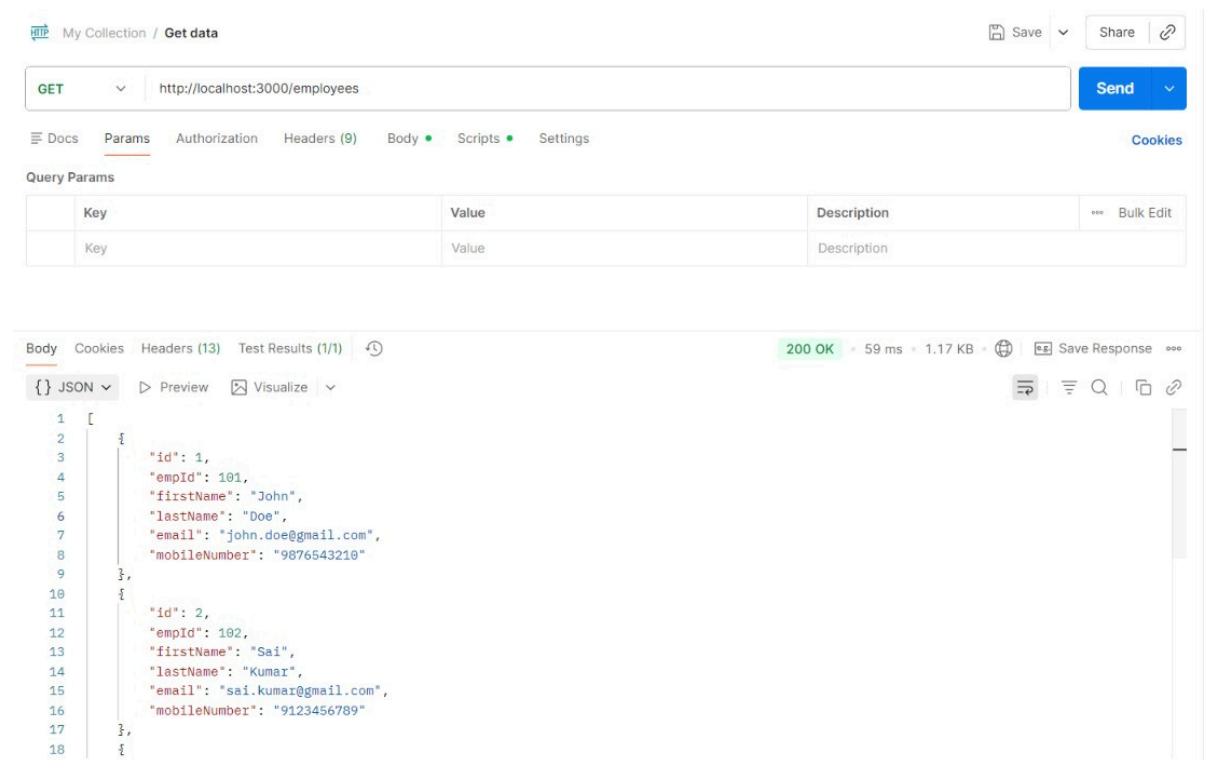
Query Params

Key	Value	Description
Key	Value	Description

Body

```
{
  "id": 1,
  "empId": 101,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@gmail.com",
  "mobileNumber": "9876543210"
},
{
  "id": 2,
  "empId": 102,
  "firstName": "Sai",
  "lastName": "Kumar",
  "email": "sai.kumar@gmail.com",
  "mobileNumber": "9123456789"
}
```

GET Method to get specific employee details



My Collection / Get data

GET http://localhost:3000/employees

Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (13) Test Results (1/1)

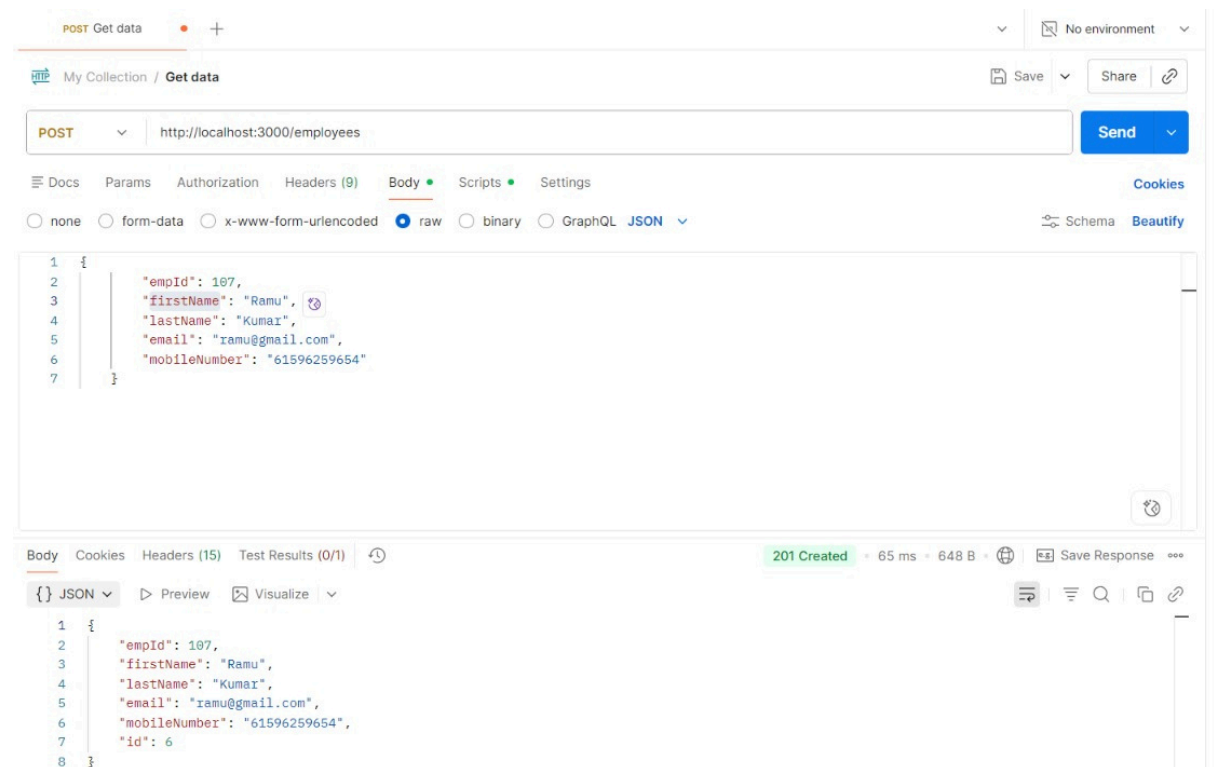
200 OK • 59 ms • 1.17 KB

Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "empId": 101,
5     "firstName": "John",
6     "lastName": "Doe",
7     "email": "john.doe@gmail.com",
8     "mobileNumber": "9876543210"
9   },
10  {
11    "id": 2,
12    "empId": 102,
13    "firstName": "Sai",
14    "lastName": "Kumar",
15    "email": "sai.kumar@gmail.com",
16    "mobileNumber": "9123456789"
17  },
18 ]
```

POST Method to Create New Employee



POST Get data

My Collection / Get data

POST http://localhost:3000/employees

Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "empId": 107,
3   "firstName": "Ramu",
4   "lastName": "Kumar",
5   "email": "ramu@gmail.com",
6   "mobileNumber": "61596259654"
7 }
```

Body Cookies Headers (15) Test Results (0/1)

201 Created • 65 ms • 648 B

Save Response

JSON Preview Visualize

```
1 {
2   "empId": 107,
3   "firstName": "Ramu",
4   "lastName": "Kumar",
5   "email": "ramu@gmail.com",
6   "mobileNumber": "61596259654",
7   "id": 6
8 }
```

PUT Method to Update Employee Details

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/employees`. The request body is a JSON object representing an employee. The response is a 201 Created status with a response body containing the updated employee details.

Request:

```
1 {
2   "empId": 107,
3   "firstName": "Ramu",
4   "lastName": "Kumar",
5   "email": "ramu@gmail.com",
6   "mobileNumber": "61596259654"
7 }
```

Response:

```
1 {
2   "empId": 107,
3   "firstName": "Ramu",
4   "lastName": "Kumar",
5   "email": "ramu@gmail.com",
6   "mobileNumber": "61596259654",
7   "id": 6
8 }
```

PATCH Method to partially update Employee Details

The screenshot shows a REST client interface with a PATCH request to `http://localhost:3000/employees/6`. The request body is a JSON object representing the partial update. The response is a 200 OK status with a response body containing the updated employee details.

Request:

```
1 {
2   "email": "ramesh@gmail.com"
3 }
```

Response:

```
1 {
2   "empId": 107,
3   "firstName": "Ramesh",
4   "lastName": "Kumar",
5   "email": "ramesh@gmail.com",
6   "mobileNumber": "61596259654",
7   "id": 6
8 }
```

DELETE Method to Delete Employee

The screenshot displays a REST client interface with the following details:

- URL:** `http://localhost:3000/employees/6`
- Method:** `DELETE`
- Body:** A JSON object:

```
{  "email": "ramesh@gmail.com"}
```
- Response:** `200 OK` with a status of `200 OK`, a response time of `277 ms`, and a size of `395 B`.
- Response Body:** A JSON object:

```
{}
```